chapter_12/01/app/javascript/contexts/venue_types.ts

```
export interface InitFromProps {
  type: "initFromProps"
  props: AppProps
}

export type VenueAction =
  | ClearHolds
  | HoldTicket
  | InitFromProps
  | SetTicketToBuy
  | SetTickets
  | UnholdTicket
```

Next, we have our first real interaction with Redux.

We call configureStore at the end of this file with our reducer to create a store and export the value to be available to other parts of the code:

chapter_12/01/app/javascript/contexts/venue_context.ts

```
export const venueStore = configureStore({ reducer: venueReducer })
export type RootState = ReturnType<typeof venueStore.getState>
export type AppDispatch = typeof venueStore.dispatch

export const useAppDispatch = () => useDispatch<AppDispatch>()
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector
```

The next four lines are some utilities that Redux enables to allow us to work with TypeScript more easily. We'll talk more about the TypeScript syntax in Chapter 13, *Validating Code with Advanced TypeScript*, but the summary of the first two lines after configureStore is that they give us new, dynamically defined, types that we can use in our code. We have RootState, which uses some fancy TypeScript that means "dynamically calculate the type of the state returned by this reducer." And we have AppDispatch, which means "dynamically calculate the type of the dispatch method used by this reducer." Finally, we use those types to augment the hooks provided by React Redux (more on that in a second when we actually use the hooks).